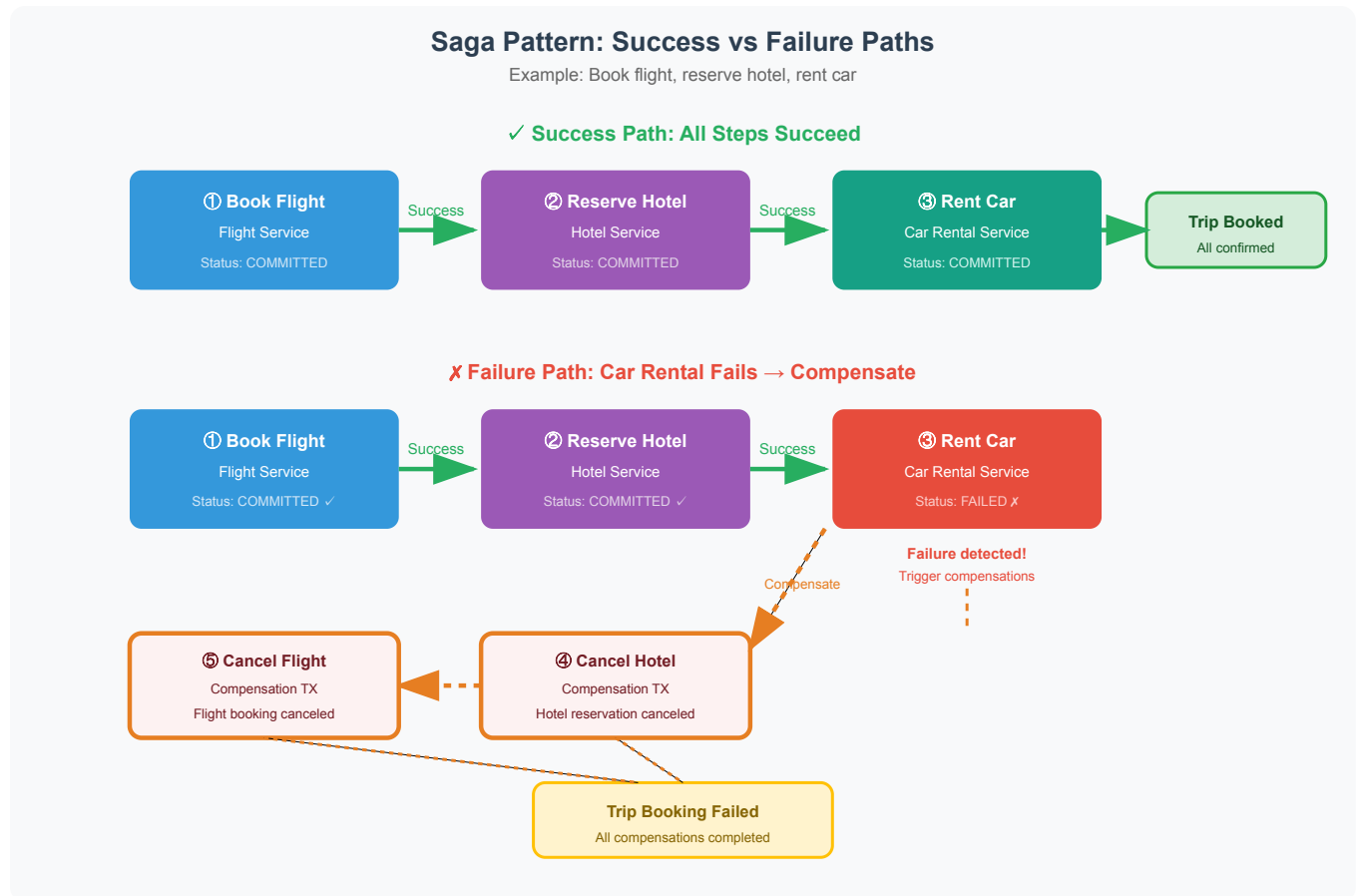


Saga Pattern

 systemdesignschool.io/fundamentals/saga-pattern



A user books a trip: reserve flight, book hotel, rent car. Three services. Three databases. The flight books successfully. The hotel reservation fails. Without coordination, the user has a flight but no hotel. The system must either complete all steps or undo completed ones. The Saga pattern coordinates multi-step operations across services using compensating transactions.

The Problem

Traditional database transactions use locks. Begin transaction, update multiple tables, commit or rollback. Either all changes apply or none do. This works within one database but fails across services.

Microservices use separate databases. The Order Service cannot lock rows in the Payment Service database. Two-Phase Commit attempts to solve this but requires blocking, adds latency, and creates availability problems. Most systems need a better approach.

How Saga Works

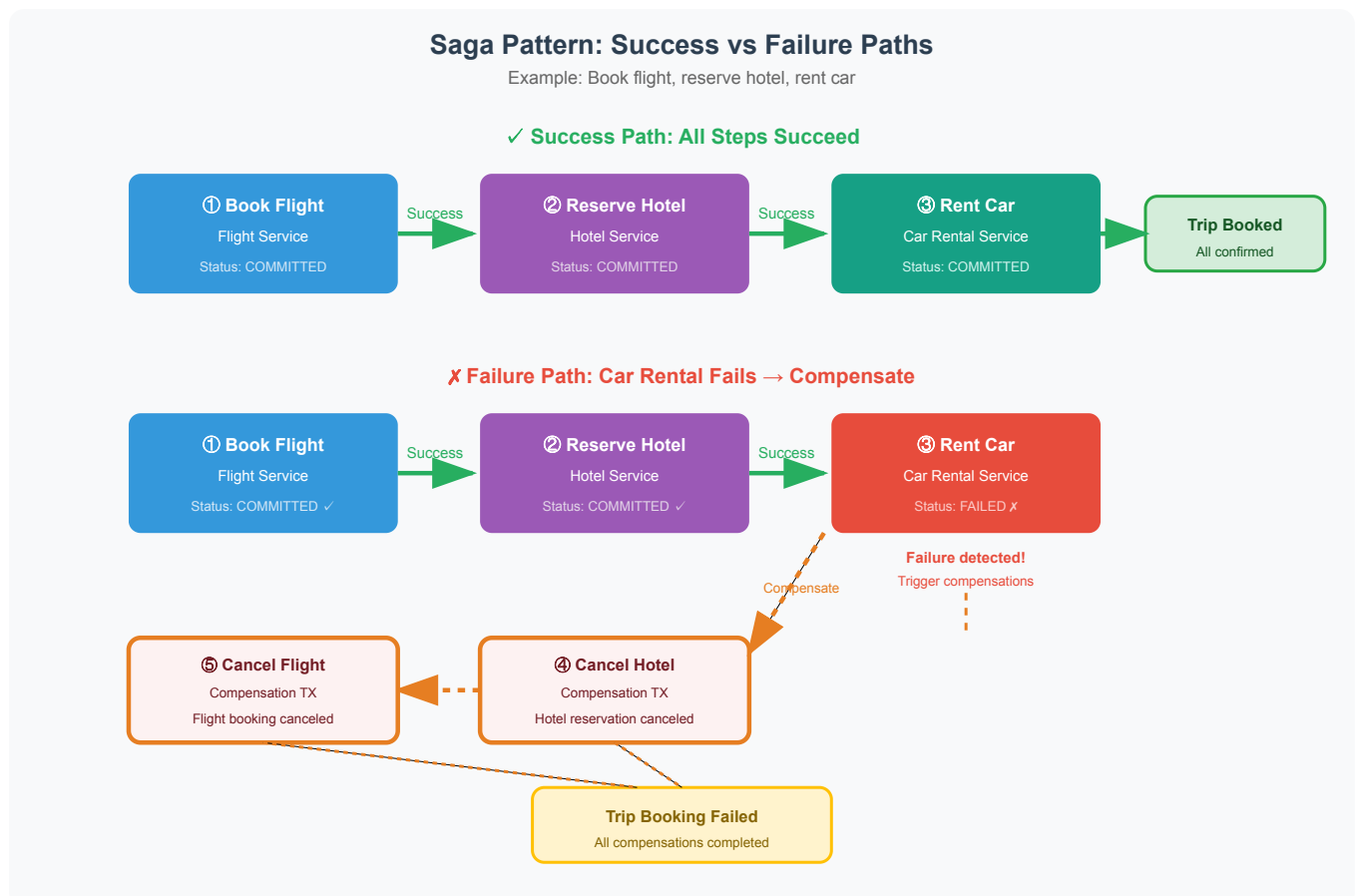
A saga breaks a transaction into local transactions, one per service. Each local transaction commits immediately to its database. If a later step fails, the saga executes compensating transactions to undo previous steps.

For the trip booking example, the success path commits each reservation:

1. Book flight → committed to flight database
2. Reserve hotel → committed to hotel database
3. Rent car → committed to rental database

The failure path uses compensating transactions:

1. Book flight → committed
2. Reserve hotel → committed
3. Rent car → **fails**
4. Cancel hotel (compensating transaction)
5. Cancel flight (compensating transaction)

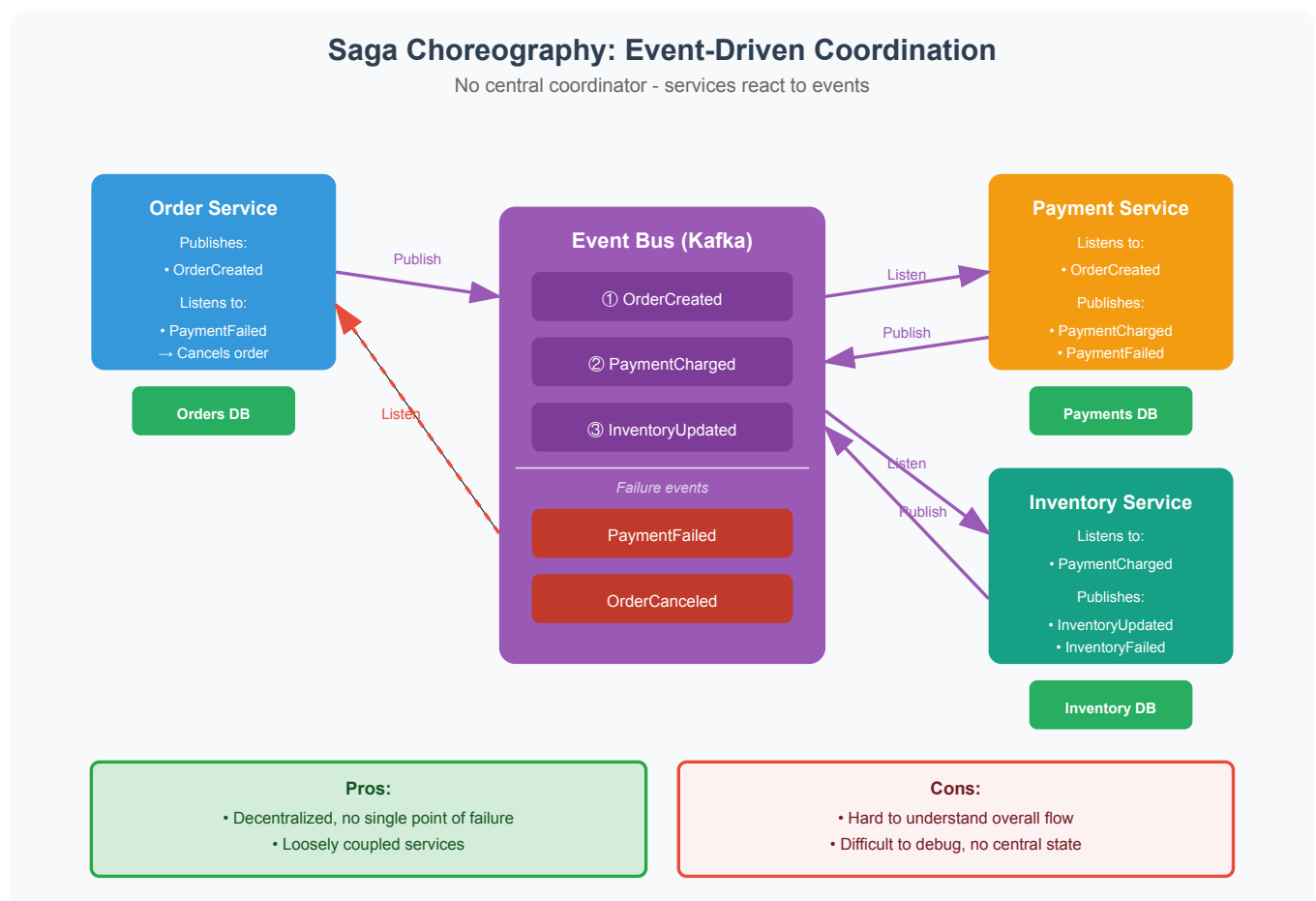


Each service provides both a forward operation and a compensating operation. The Flight Service exposes `bookFlight` and `cancelBooking`. The Hotel Service exposes `reserveRoom` and `cancelReservation`. When a saga step fails, the saga executor calls compensating operations in reverse order.

Implementation Approaches

Sagas implement through choreography or orchestration.

Choreography uses events. Services listen for events and react. The Order Service creates an order and publishes "OrderCreated". Payment Service listens, charges payment, publishes "PaymentCharged". Inventory Service listens, decrements stock, publishes "InventoryUpdated". If payment fails, Payment Service publishes "PaymentFailed". Order Service listens and cancels the order.



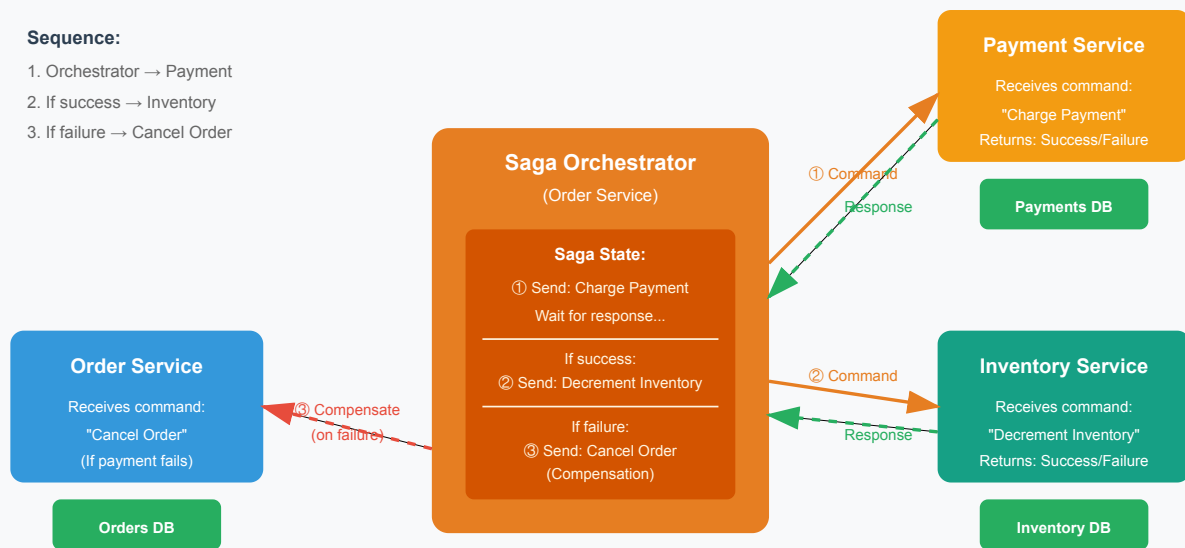
Orchestration uses a central coordinator. The Order Service acts as orchestrator. It calls Payment Service, waits for response. If payment succeeds, it calls Inventory Service. If inventory fails after payment succeeds, it calls Payment Service to issue a refund.

Saga Orchestration: Centralized Coordination

Central orchestrator manages the saga flow

Sequence:

1. Orchestrator → Payment
2. If success → Inventory
3. If failure → Cancel Order



Pros:

- Clear flow, easy to monitor
- Centralized state management

Cons:

- Orchestrator is a dependency
- Can be stateless & replicated



Orchestration is simpler for most systems. The flow lives in one place. Debugging follows a clear path. State management centralizes. Use choreography only when maximum decoupling justifies the complexity.

Trade-offs

Sagas provide availability at the cost of temporary inconsistency. After step 1 completes, the system has a booked flight but no hotel yet. This state is visible to other operations. The user might see their flight in "My Trips" before the hotel books or before compensating transactions cancel everything.

This eventual consistency is acceptable for most business operations. Users understand "processing" states. The booking completes within seconds. Clear communication makes the temporary inconsistency manageable.

Compensating transactions are not true rollbacks. A canceled flight booking creates a cancellation record. The system cannot erase the fact that a booking existed. Compensation must handle idempotency. If the system retries a cancellation, calling `cancelBooking` twice with the same booking ID must work correctly.

Interview Application

E-commerce checkout provides a clear saga example. A user checks out with three operations: create order, charge payment, decrement inventory.

Using orchestration, the Order Service coordinates the saga. It creates the order record first. Then it calls Payment Service. If payment succeeds, it calls Inventory Service. If inventory fails after payment, it calls Payment Service to refund.

Explain the compensating transactions explicitly. Each service must expose compensation endpoints. Payment Service has `chargePayment` and `issueRefund`. Inventory Service has `decrementStock` and `returnStock`. These compensations are business operations, not database rollbacks.

Discuss failure scenarios. What if the network fails while calling a compensation? The orchestrator must retry until it succeeds. Compensations must be idempotent. Use unique idempotency keys to prevent duplicate refunds.

Mention eventual consistency clearly. After the order creates, it appears in the order history. If payment fails 2 seconds later, the order transitions to "failed" state. Users see this transition. Clear status messages prevent confusion.

For deeper understanding of distributed transactions, consistency guarantees, and comparisons with Two-Phase Commit, see [Distributed Transactions](#).

When to Use

Use sagas when operations span multiple services with separate databases, availability matters more than immediate consistency, and you can design compensating transactions for each operation. E-commerce checkout, travel booking, and order fulfillment workflows fit this pattern.

Avoid sagas when operations require perfect consistency at all times, compensating transactions cannot truly undo the operation, or the business logic cannot tolerate temporary inconsistent states visible to users.

Most modern distributed systems use sagas. The pattern balances coordination needs with performance and availability requirements.